

```

import gymnasium as gym
import torch
import torch.optim as optim
import torch.nn as nn
import random
from collections import deque
import numpy as np

# 신경망 구조 코딩(선형)
class DQN(nn.Module):

    def __init__(self, state_dim, action_dim):# state_dim = 상태 크기(변수 갯
수), action_dim = 행동 크기(행동 가짓수)
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim)
        )

    def forward(self, x):
        return self.net(x)

# Replay Buffer코딩(Replay Buffer : 과거 경험 저장 -> 나중에 사용 가능하게 해서 효율성
높임
class Replay_Buffer:

    def __init__(self, capacity = 10000):
        self.buffer = deque(maxlen = capacity)

    def push(self, s, a, r, s_next, done):
        self.buffer.append((s, a, r, s_next, done)) # s:현재 상태, a:행동, r:
보상, s_next:다음 상태, done:종료 여부

    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        s, a, r, s_next, done = zip(*batch)
        return s, a, r, s_next, done

    def __len__(self):
        return len(self.buffer)

env = gym.make("CartPole-v1")

```

```

state_dim = env.observation_space.shape[0]
action_dim = env.action_space.n
policy_net = DQN(state_dim, action_dim) # 실제 학습용 신경망
# policy_net.load_state_dict(torch.load("best.pth")) # 최신 좋은 모델 불러오기
target_net = DQN(state_dim, action_dim) # 계산/안전성용 신경망
target_net.load_state_dict(policy_net.state_dict()) # 초기 상태는 동일하게
target_net.eval() # 애는 학습 안함
optimizer = optim.Adam(policy_net.parameters(), lr = 1e-3)
buffer = Replay_Buffer()
gamma = 0.99 # 할인율
# 입실론-그리디 탐색 -> 가끔 튀는 행동 해서 최적 전략 찾을 확률 높이기
epsilon = 1.0
epsilon_decay = 0.999
epsilon_min = 0.01
batch_size = 64
target_update = 10
episodes = 25000

for ep in range(episodes):
    obs, _ = env.reset()
    state = torch.tensor(obs, dtype = torch.float32)
    done = False
    total_reward = 0
    while not done:
        if np.random.rand() < epsilon:
            action = env.action_space.sample()
        else:
            with torch.no_grad():
                action = policy_net(state).argmax().item()
        next_obs, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        next_state = torch.tensor(next_obs, dtype=torch.float32)
        buffer.push(state, action, reward, next_state, done)
        state = next_state
        total_reward += reward
        # 학습 (데이터 충분할 때)
        if len(buffer) >= batch_size:
            s, a, r, s_next, d = buffer.sample(batch_size)
            s = torch.stack(s)
            a = torch.tensor(a).unsqueeze(1)
            r = torch.tensor(r, dtype=torch.float32)
            s_next = torch.stack(s_next)
            d = torch.tensor(d, dtype=torch.float32)
            q = policy_net(s).gather(1, a).squeeze()
            with torch.no_grad():
                q_next = target_net(s_next).max(1)[0]
                target = r + gamma * q_next * (1 - d)
            loss = nn.MSELoss()(q, target)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        epsilon = max(epsilon_min, epsilon * epsilon_decay)
    if ep % target_update == 0:
        target_net.load_state_dict(policy_net.state_dict())

```

```
    print(f"Episode {ep + 1}, Reward {total_reward}")
env.close()

env = gym.make("CartPole-v1", render_mode = "human", max_episode_steps =
1000000000)

state_dim = env.observation_space.shape[0]
action_dim = env.action_space.n
episodes = 10

for ep in range(episodes):
    obs, _ = env.reset()
    state = np.array(obs, dtype = np.float32).reshape(1, 4)
    done = False
    total_reward = 0
    while not done:
        # Core ML 모델로 action 선택
        with torch.no_grad():
            action_values = policy_net(torch.tensor(state,
dtype=torch.float32))
            action = int(action_values.argmax().item()) # 최대 Q-value 선택

        # 환경 진행
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        state = np.array(next_state, dtype=np.float32).reshape(1, 4)
        total_reward += reward

    print(f"Episode {ep+1}: Total Reward = {total_reward}")

env.close()

save = input("Save model? (y/n)\n")
if save.lower() == "y":
    torch.save(policy_net.state_dict(), input("Enter model name :
")+ ".pth")
else:
    print("Model not saved.")
```